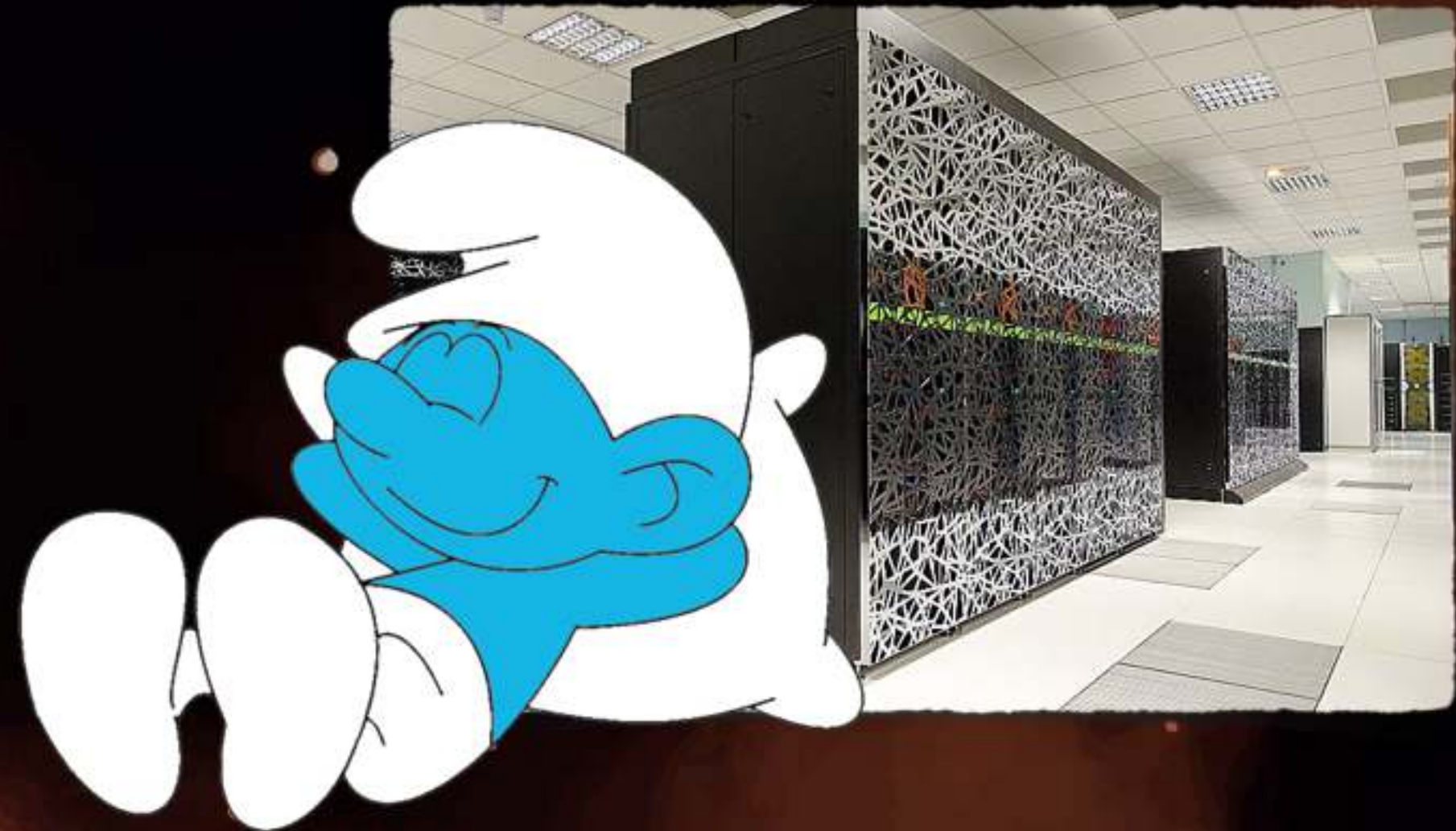


De la notion de «macros» à la notion de généricité

Petite pause...
...sur la concurrence



Fabrice.Kordon@lip6.fr

En guise d'introduction...

2

La notion de macro

Substituer une séquence de texte compliquée à une séquence plus simple

- Premières utilisations en assembleur
 - ▶ Assimiler une séquence de code compliquée à quelque chose de plus simple
 - ▶ Usage de paramètres
- C dispose de cette fonctionnalité via le pré-processeur

Exemple, jouer avec la syntaxe de C

3

```
/* Petit exemple sur l'usage de macro */  
#include <stdio.h>
```

```
/* redefinition (limitee) du langage */  
#define Si if (  
#define Alors ){  
#define FinSi }  
#define Sinon } else {  
#define SinonSi } else if (
```

Code généré par le pré-processeur (cpp)

```
$ cpp ex-macro1.c
```

... le contenu de `stdio.h` ...

```
# 10 "ex-macro1.c"
```

```
int main ()  
{  
    int a;  
  
    printf ("entrer une valeur\n");  
    scanf ("%i", &a);  
    printf ("vous avez saisi la valeur %i\n",  
a);
```

```
int main () {  
    int a;  
  
    printf ("entrer une valeur\n");  
    scanf ("%i", &a);  
    printf ("vous avez saisi la valeur %i\n",  
a);
```

```
    Si a < 0 Alors  
        printf ("a est negatif\n");  
    SinonSi a == 0 Alors  
        printf ("a est nul\n");  
    Sinon  
        printf ("a est positif\n");  
    FinSi  
}
```

```
if ( a < 0 ){  
    printf ("a est negatif\n");  
} else if ( a == 0 ){  
    printf ("a est nul\n");  
} else {  
    printf ("a est positif\n");  
}  
}
```


Macros avec paramètres

4

```
/* Autre petit exemple sur l'usage de macro */
```

```
#include <stdio.h>
```

```
/* macro avec des paramètres */
```

```
#define max(a,b) (a)>(b)?(a):(b)
```

```
int main ()
```

```
{
```

```
    int a, b;
```

```
    printf ("entrer une valeur\n");
```

```
    scanf ("%i", &a);
```

```
    printf ("entrer une autre valeur\n");
```

```
    scanf ("%i", &b);
```

```
    printf ("%i est le max entre %i et %i\n", max (a,b), a, b);
```

```
}
```

Macros avec paramètres

4

Code généré par le pré-processeur (cpp)

```
$ cpp ex-macro2.c
```

... le contenu de *stdio.h* ...

```
# 6 "ex-macro2.c"
int main ()
{
    int a, b;

    printf ("entrer une valeur\n");
    scanf ("%i", &a);
    printf ("entrer une autre valeur\n");
    scanf ("%i", &b);
    printf ("%i est le max entre %i et %i\n", (a)>(b)?(a):(b), a, b);
}
```


Observations sur les macros

5

Mécanisme extrêmement utile

- Substitution de «séquences types»
- Centralisation de modifications
 - ▶ Dimensionnement de constantes
 - ▶ Réécriture de séquences d'instructions

Limites du système

- Difficile de vérifier ce qui est substitué
 - ▶ Le compilateur travaille sur le code réécrit (c'est trop tard)
- Impossible de donner un cadre strict aux substitutions
 - ▶ Spécifier qu'une constante doit être d'un type donné
 - ▶ Spécifier le prototype d'une fonction à remplacer
- ... bref, peu de contrôle au niveau de l'expansion de macro-définitions

Vers la généricité

6

Notion évoluée de macro-génération

- Avec contrôle au niveau des «paramètres génériques formels»

Liée à la notion de «comportement modèle»

- Traitement général approprié à un ensemble de «concepts»
- Les «concepts» sont caractérisés par
 - ▶ Des valeurs
 - ▶ Des types
 - ▶ Des primitives
- Le «comportement modèle» n'est pas utilisable en tant que tel

La notion de «modèle» — un mécanisme sûr

7

Échanger le contenu de deux boîtes.

- Prendre une «boîte intermédiaire»
- Mettre le contenu de la première boîte dans la «boîte intermédiaire»
- Mettre le contenu de la seconde boîte dans la première boîte
- Mettre le contenu de la «boîte intermédiaire» dans la seconde boîte

La démarche est valable dans de nombreux cas de figure

- Échanger le contenu de deux verres d'eau
- Échanger le contenu de deux disquettes
- Échanger le contenu de deux variables

Aspect générique $\Rightarrow$ le contenu de la boîte

La notion de «modèle» — un mécanisme sûr

7



Différences avec les macros

Un contrôle du typage plus fin
Classification (types, valeurs, etc.)

Échange

- Prendre un
- Mettre le c
- Mettre le c
- Mettre le contenu de la «boîte intermédiaire» dans la seconde boîte



Le coin culture...

Première implémentation en Ada (83)

La déma figure

- Échanger le
- Échanger le
- Échanger le



Et avant?

Macros et/ou pointeurs dans tous les sens!

Aspect générique → le contenu de la boîte

En guise de conclusion

8

C'est un mécanisme devenu «classique»

-  Cohabitation possible avec le modèle objet
-  Ada (generics)
-  C++ (templates)
-  Java (1.5+, generics)
-  etc.

Nous en aurons besoin pour certains utilitaires

-  Étudions le!